

Informática Musical

Ejercicios. Hoja 5

Todos los ejercicios de esta hoja son evaluables, pero **solo es obligatorio uno, a elegir por el alumno**. Es recomendable hacer alguno más.

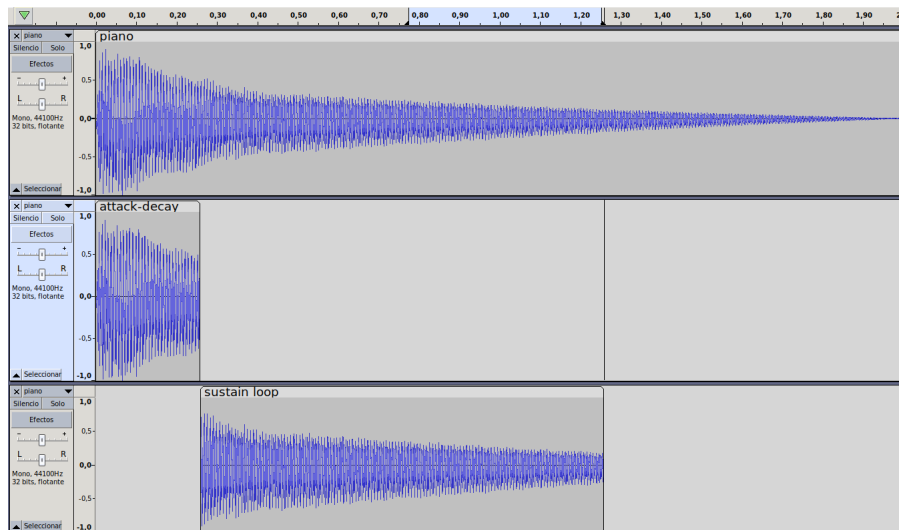
La entrega debe ser autocontenido (incluir todo lo necesario para ejecutar el notebook). Se realizará en formato Jupyter Notebook, con comentarios explicativos. El notebook debe contener solo las celdas específicas para cada ejercicio. El resto de módulos se incorporarán con *includes* en dicha celda, que se adjuntarán en archivos independientes *.py*.

1. Implementar un *sampler* simple utilizando la muestra *piano.wav*. Mapear las teclas *zxc...* a una octava del piano y *qwe...* a la octava superior. A partir de muestra de una nota el resto de notas pueden obtenerse variando la velocidad de reproducción de esa muestra de acuerdo a estas proporciones. Por ejemplo, asumiendo que la nota dada es un C, el resto de notas pueden obtenerse con las siguientes proporciones¹:

Nota	C	D	E	F	G	A	B	c	d ... a
Frec.	1	9/8	5/4	4/3	3/2	5/3	15/8	2	...

Es decir, para obtener un D (re) reproduciríamos el sample (virtualmente) a 1.12 de velocidad. Observemos que dado un array de *NumPy*, si generamos otro que contenga solo los samples en posición par (de la mitad de tamaño), obtenemos un sonido una octava por arriba. Y al contrario, si generamos un array del doble de tamaño añadiendo muestras intermedias (como media de la anterior y la superior de una dada) obtenemos una octava menos. Esta idea de interpolación puede generalizarse para obtener cualquier nota, utilizando las proporciones dadas. Esto, a su vez puede hacerse de manera explícita o utilizando primitivas de *numPy* o *scipy* (como **resample**).

2. Implementar un *sampler* que permita cargar muestras de audio *preparadas* para procesar en tiempo real. En particular, para una nota de piano como la de la primera pista de abajo, podemos utilizar dos fragmentos. El primero con la parte de ataque y decay, y el segundo con una zona *loopeable* para construir el sustain (la última parte de la muestra inicial la podemos descartar).



El fragmento de sustain debe elegirse adecuadamente para poder construir el loop. Sobre ese fragmento hay que linealizar el volumen y asegurarse de que suena bien cíclicamente.

Estos dos fragmentos serán los que cargue el sampler. El mensaje *noteOn* activará la generación de una nota: se reproduce la región de ataque-decay y luego se reproduce en loop la región de sustain

¹Afinación justa, también puede hacer con temperada, tal como se hizo en la hoja anterior

hasta que llegue el mensaje *noteOff*. En ese momento se aplica un release a la región sustain (con tiempo parametrizable) hasta silenciar la nota.

Para definir el pitch de la nota, se tomará el pitch de la nota original y se variará la velocidad de reproducción para conseguir la nota deseada. Puede utilizarse la técnica de wavetable (recorrido a distintas velocidades variando el *step* de avance y utilizando interpolación), o bien, remuestreo de señal (función de numpy).

3. Utilizar el filtro IIR visto en clase para implementar un filtro paso banda (BP) con frecuencia de corte y ancho de banda configurables. Utilizar dos filtros LP y HP en secuencia, calculando los valores de α según se ha explicado en clase.
4. Utilizando la filosofía de la clase **Instrument** vista en clase, implementar un instrumento **piano** con el modelo de síntesis de Karplus-Strong. Los chunks del método **next()** se obtendrán a partir del buffer circular que incorpora el algoritmo.

Para investigar: la afinación se deriva de la longitud (en samples) del buffer de partida, lo que implica que pueden conseguirse frecuencias discretas. ¿Es posible conseguir una afinación más precisa, no restringida a ese número de samples?